



UNIMORE

UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITÀ DEGLI STUDI DI MODENA
E REGGIO EMILIA

Dipartimento di Ingegneria "Enzo Ferrari"
Corso di Laurea Triennale in Ingegneria Informatica (L-08)

Traffic Shaping in User Space: Fattibilità e Limiti

Tutor:

Prof. Riccardo Lancellotti

Candidato:

Leonardo Donati
Matricola 191271

Indice

1	Introduzione	5
2	Traffic Shaping	6
2.1	Definizione	6
2.2	Traffic Shaping in Ambienti Linux	6
2.2.1	Queueing Discipline (qdisc)	6
2.2.2	Token Bucket Filter (TBF)	7
2.2.3	Hierarchical Token Bucket (HTB)	8
2.3	Esempi Reali di Utilizzo	10
2.3.1	Limitazione Banda su Backup	10
2.3.2	Server con Multipli Servizi	10
3	Kathará	12
3.1	Introduzione	12
3.2	Architettura di Kathará	12
3.3	Configurazione	13
3.3.1	Riproduzione dello scenario HTB in Kathará	14
3.4	Dipendenza dai moduli del kernel host	16
4	Traffic Shaping in Userspace	17
4.1	Iptables	17
4.1.1	Target NFQUEUE	18
4.1.2	Netlink	19
4.1.3	Libnetfilter_queue	19
4.2	Damper	20
4.2.1	Logica di funzionamento	20
4.2.2	Configurazione	21
5	Casi di Studio	23
5.1	Introduzione	23
5.2	Limitazione tra due Nodi	24
5.2.1	Soluzione	24
5.2.2	Risultati	25
5.3	Limitazione Differenziata verso più Nodi	26
5.3.1	Soluzione	26
5.3.2	Risultati	27
5.4	Limitazione Banda con Gateway/Router	28
5.4.1	Soluzione	29
5.4.2	Risultati	29
5.5	Limiti di Damper	30
6	Donper	31
6.1	Introduzione	31
6.2	Configurazione dei Parametri	32
6.2.1	Struttura Dati	32
6.3	Classificazione dei Pacchetti	34
6.4	Gestione della Trasmissione di Pacchetti	35
6.4.1	Prestito fra classi	35
6.4.2	Gestione senza Classi	37
6.5	Verifica Sperimentale	37

<i>INDICE</i>	3
6.5.1 Soluzione	38
6.5.2 Risultati	39
6.6 Considerazioni Finali	39
7 Conclusione	40

Elenco delle Figure

2.1	Funzionamento di TBF	7
2.2	Gerarchia HTB	8
2.3	Classificazione pacchetti con filtri in HTB.	9
2.4	Rappresentazione grafica del primo scenario.	10
2.5	Rappresentazione grafica del secondo scenario.	10
3.1	Architettura di Kathará [7]	12
3.2	Docker Container vs Virtual Machine	16
4.1	Rappresentazione della situazione descritta.	18
4.2	Ciclo di vita pacchetto NFQUEUE.	19
4.3	Funzionamento di Damper.	20
5.1	Primo scenario.	24
5.2	Secondo scenario.	26
5.3	Terzo scenario.	28
6.1	Architettura generale Donper.	31
6.2	Rappresentazione grafica scenario.	37

Capitolo 1

Introduzione

Il traffic shaping è una tecnica ampiamente utilizzata nelle reti informatiche per controllare la larghezza di banda disponibile a utenti, servizi o flussi di traffico specifici. In ambienti Linux, è possibile tramite il comando `tc`, che agisce direttamente all'interno del kernel utilizzando diverse queueing discipline.

Questo approccio presenta un limite in ambienti containerizzati: non disponendo di un kernel proprio, un container usa le funzionalità di rete messe a disposizione dal kernel dell'host. L'assenza dei moduli necessari al comando `tc` può quindi creare problemi con portabilità e riproducibilità di uno scenario di rete in diversi sistemi.

L'obiettivo di questa tesi è verificare la fattibilità di un approccio alternativo, che sposti la logica di traffic shaping dal kernel allo spazio utente con il framework Netfilter e le code NFQUEUE.

La tesi è strutturata come segue: il capitolo 2 introduce il traffic shaping in ambienti Linux, con le discipline di accodamento TBF e HTB. Il capitolo 3 mostra il funzionamento di Kathará, il programma usato per le simulazioni degli scenari di rete e i suoi limiti. Il capitolo 4 spiega l'approccio di traffic shaping in userspace e del framework Netfilter. Il capitolo 5 ne dimostra la fattibilità tramite il programma Damper, evidenziato i suoi limiti strutturali. Infine, il capitolo 6 descrive in modo dettagliato il fork Donper, che permette di gestire classi e condivisione di banda in modo simile, ma non uguale, alla qdisc spiegate nel capitolo 1.

Capitolo 2

Traffic Shaping

2.1 Definizione

DEFINIZIONE Traffic Shaping

Il traffic shaping è una tecnica di gestione della larghezza di banda utilizzata nelle reti informatiche, che ritarda alcuni o tutti i datagrammi IP per adeguarli al profilo di traffico desiderato. Consente di migliorare le prestazioni della rete controllando in il traffico in entrata e in uscita o di simulare condizioni di reti reali.

A differenza di tecniche basate sullo scarto di datagrammi IP in eccesso, il traffic shaping ne ritarda l'invio, trattendoli in una coda.

Esistono alcuni scenari tipici in cui è applicato:

- **Limitazione** della banda per utenti e servizi in una rete;
- **Garanzia** di banda minima per determinati flussi;
- **Prioritizzazione** del traffico;
- **Minimizzazione** della latenza;

2.2 Traffic Shaping in Ambienti Linux

In ambienti Linux il traffic shaping può essere effettuato con il comando `tc` (traffic control) [1], disponibile nella suite di comandi `iproute2` [2].

Nella pagina ufficiale del manuale di [1], viene evidenziato come l'elaborazione del traffico è gestita tramite tre tipi di oggetti: **qdisc**, **classi** e **filtri**.

2.2.1 Queueing Discipline (qdisc)

DEFINIZIONE qdisc

qdisc è l'abbreviazione di queueing discipline: rappresenta il meccanismo con cui il kernel gestisce l'invio dei pacchetti su un'interfaccia, accodandoli secondo una politica configurabile [1].

Alcune qdisc possono essere **classful**, quindi contenere classi con a loro volta una propria qdisc, creando di fatto una gerarchia. Il kernel può decidere di inviare un pacchetto di una classe invece che un'altra, per dare priorità a determinati tipi di traffico cercando di estrarre pacchetti prima da certe classi.

Eventualmente, le qdisc possono essere pure **classless**, potendo solo riordinare, ritardare o scartare pacchetti.

2.2.2 Token Bucket Filter (TBF)

DEFINIZIONE Token Bucket Filter

Disciplina di accodamento **classless** disponibile per il controllo del traffico nel comando **tc**. Funziona puramente da shaper e non si occupa mai dello scheduling del traffico. [3]

TBF è molto leggero e non appesantisce la rete né il processore. Il suo unico limite è che può lavorare su una interfaccia usando una sola coda.

Algoritmo

L'implementazione consiste in un buffer (**bucket**) che viene costantemente riempito di gettoni (**tokens**), a una velocità specifica **rate**. Il bucket ha una capienza (**burst**), che indica quanti gettoni può contenere al massimo. Un token equivale a circa un byte.

Quando un pacchetto deve essere inviato, i gettoni necessari per inviarlo vengono rimossi dal bucket. Nel caso non fossero sufficienti, il pacchetto viene inserito in una coda finché non vengono generati abbastanza gettoni per inviarlo, in caso contrario viene mandato immediatamente. Sfruttando il rate dei gettoni e il burst è possibile, quindi, ritardare l'invio dei pacchetti.

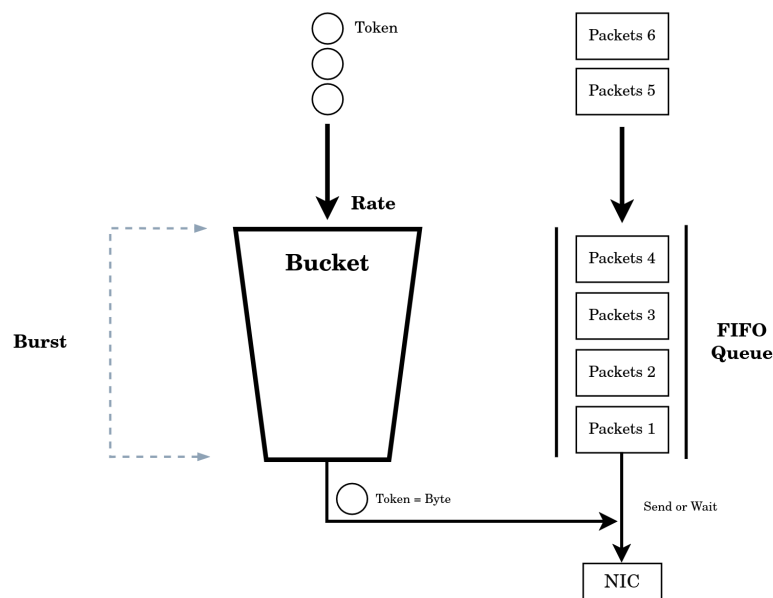


Figura 2.1: Funzionamento di TBF

La coda dei pacchetti utilizza una politica **FIFO** (First In, First Out) e la sua grandezza in byte può essere decisa tramite il parametro **limit**. Se piena, i pacchetti successivi vengono scartati direttamente.

Parametri comando tc per TBF [3]		
Parametro	Unità di Misura	Descrizione
rate	Mbit/s	Velocità di generazione dei tokens.
burst	Byte	Capienza del bucket.
limit	Byte	Capienza della coda.

2.2.3 Hierarchical Token Bucket (HTB)

DEFINIZIONE Hierarchical Token Bucket

Disciplina di accodamento **classful** disponibile per il controllo del traffico nel comando `tc`. Utilizzato per fare traffic shaping controllando la banda in uscita di un'interfaccia, permettendo di dividere vari flussi tramite l'uso di un albero per creare una **gerarchia di classi**. L'algoritmo utilizzato per l'elaborazione del traffico è token bucket filter [4].

Ogni classe rappresenta un nodo di un **albero** e, ad eccezione della **radice**, ha una classe **genitore**. Questa struttura permette di suddividere ricorsivamente la banda assegnata a una classe tra le sue classi figlie, ciascuna configurabile con i propri parametri: **rate** e **ceil**.

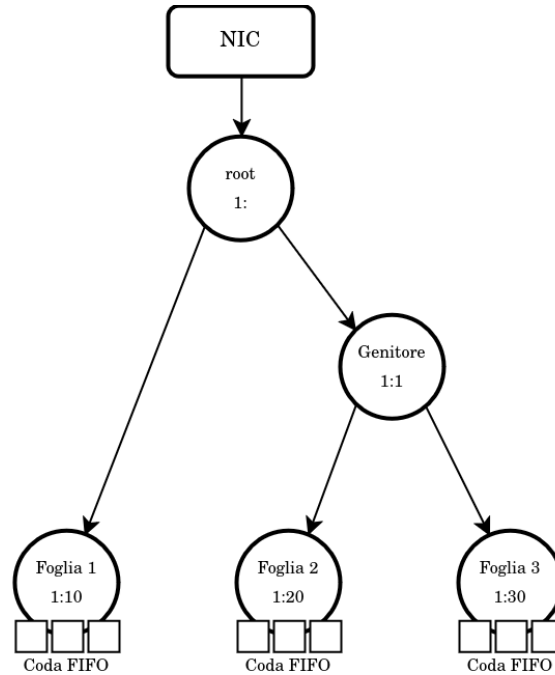


Figura 2.2: Gerarchia HTB

Le code FIFO di pacchetti in attesa possono essere solo sulle foglie dell'albero, i nodi intermedi non possono accodare.

Algoritmo di Condivisione della Banda

Tramite questo algoritmo, quando una classe necessita di più banda di quella garantita dal proprio rate, può prendere in prestito la capacità non utilizzata dalle classi che condividono lo stesso genitore, fino al limite imposto dal proprio ceil. Il parametro rate indica la banda **minima** garantita, invece il parametro ceil indica la banda **massima** utilizzabile.

Una classe genitore conosce la somma della banda effettivamente utilizzata dalle proprie classi figlie e può quindi ridistribuirle a quelle che ne richiedono di più. Considerando che come algoritmo viene usato token bucket, possiamo includere tutti i casi possibili dei prestiti in tre situazioni diverse [5]:

- Il numero di tokens di una classe è sufficiente per inviare immediatamente il pacchetto.
- Il numero di tokens di una classe non è sufficiente per inviare immediatamente un pacchetto, ma è possibile chiedere un prestito alla classe genitore, visto che la banda non è saturata.
- Il numero di tokens di una classe non è sufficiente per inviare immediatamente un pacchetto e non è possibile prendere in prestito ulteriore banda. Il pacchetto viene aggiunto alla coda finché non sarà possibile spedirlo.

I parametri utili per l'algoritmo di prestito sono:

Parametro	Unità di Misura	Descrizione
rate	Mbit/s	Velocità minima garantita di generazione dei tokens.
burst	Byte	Capienza del bucket.
ceil	Mbit/s	Velocità massima possibile di generazione dei tokens.
cburst	Byte	Capienza del bucket necessaria quando la generazione dei tokens è al massimo possibile.
quantum	Byte	Byte da inviare prima di passare a un'altra classe.

Filtri

DEFINIZIONE Filtro

Metodo usato per inserire il pacchetto in una determinata classe in base a diverse caratteristiche, è possibile farlo pure tramite il comando `tc` (traffic control) [1]. Non è l'unico modo pensato per classificare i pacchetti, possono essere usate pure le iptables [6].

I filtri sono l'oggetto che lega le code di pacchetti alle classi e possono farlo secondo vari criteri: ip, porte, protocolli, fwmark nel caso vengano usate le iptables e molti altri.

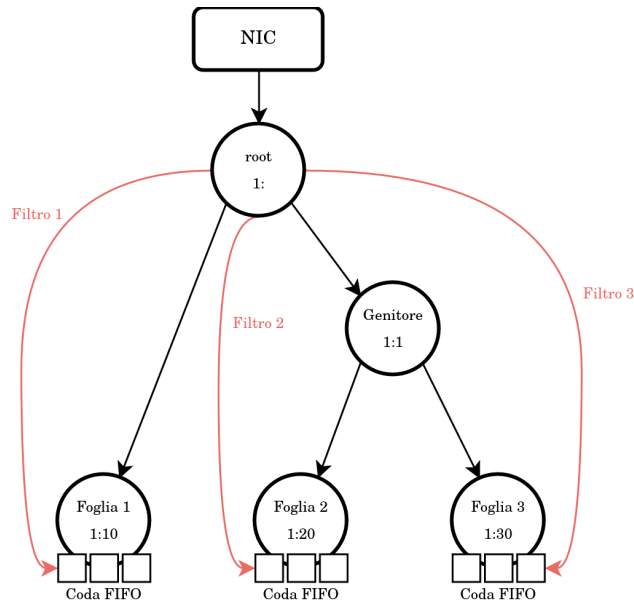


Figura 2.3: Classificazione pacchetti con filtri in HTB.

Normalmente la classificazione avviene una sola volta, con i filtri applicati al nodo radice, ma è possibile aggiungere ulteriori filtri nei nodi intermedi.

2.3 Esempi Reali di Utilizzo

Il traffic shaping è usato anche in contesti reali. Nei prossimi paragrafi, vorrei mostrare alcuni casi d'uso in cui è utile applicare discipline di accodamento come Token Bucket Filter o Hierarchical Token Bucket.

2.3.1 Limitazione Banda su Backup

SCENARIO

Un piccolo ufficio ha una connessione internet condivisa da 20 Mbit/s in upload. Un server esegue un backup automatico verso il cloud in diversi orari della giornata e se lasciato libero satura completamente la banda in upload. Si vuole limitare la velocità del traffico in uscita in modo da permettere ai dipendenti di lavorare tranquillamente.

Server Backup

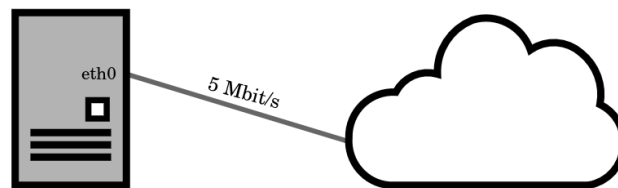


Figura 2.4: Rappresentazione grafica del primo scenario.

In questo caso, è possibile limitare il traffico sull'interfaccia di rete del server di backup. Ipotizziamo che l'interfaccia sia `eth0` e che si voglia limitare il traffico a 5 Mbit/s:

BASH Comando `tc`

```
tc qdisc add dev eth0 root tbf rate 5mbit burst 32kb limit 100kb
```

Ora, la connessione è correttamente limitata.

2.3.2 Server con Multipli Servizi

SCENARIO

Un server di un'azienda ospita tre servizi per tre clienti diversi: un servizio streaming, un sito web con molto traffico e un servizio di file hosting.

La banda totale disponibile e garantita è 100 Mbit/s. Si vuole garantire più banda al cliente del sito web rispetto agli altri due clienti. Gli altri due servizi avranno un minimo di banda garantita.

Server

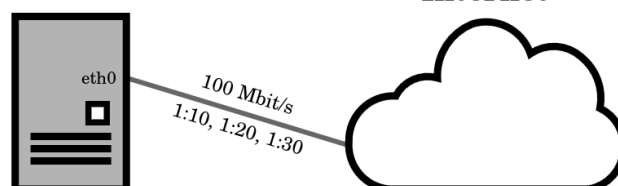


Figura 2.5: Rappresentazione grafica del secondo scenario.

In questo caso, con opportuni filtri è possibile inserire i pacchetti nelle code delle classi corrette: 1:10 per il sito web, 1:20 per lo streaming e 1:30 per il file hosting.

BASH Classi e parametri con comando tc

```
# Nodo root e genitore
tc qdisc add dev eth0 root handle 1: htb default 30
tc class add dev eth0 parent 1: classid 1:1 htb rate 100mbit ceil 100mbit

# Classe sito web
tc class add dev eth0 parent 1:1 classid 1:10 htb rate 50mbit ceil 100mbit

# Classe servizio streaming
tc class add dev eth0 parent 1:1 classid 1:20 htb rate 35mbit ceil 60mbit

# Classe servizio file hosting
tc class add dev eth0 parent 1:1 classid 1:30 htb rate 15mbit ceil 20mbit
```

Il nodo genitore ha un minimo e un massimo di banda di 100Mbit/s, che dividerà fra le tre classi, permettendo pure il prestito nel caso fosse possibile.

L'ultima cosa che rimane da fare è scrivere i vari filtri che divideranno il traffico nelle opportune code.

BASH Filtri con comando tc

```
# Filtri per sito web
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 80 0xffff
↪ flowid 1:10
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 443
↪ 0xffff flowid 1:10

# Filtro per servizio streaming
tc filter add dev eth0 protocol ip parent 1:0 prio 2 u32 match ip sport <port>
↪ 0xffff flowid 1:20

# Filtro per servizio file hosting
tc filter add dev eth0 protocol ip parent 1:0 prio 3 u32 match ip sport <port>
↪ 0xffff flowid 1:30
```

In questo modo, il traffico in uscita con porta sorgente 80 e porta 443 (http e https), andrà correttamente nella classe creata per il sito web. I pacchetti degli altri due servizi verranno classificati, come fatto in precedenza, in base alla loro porta sorgente.

Nel comando è stata inserita una diversa priorità per la valutazione dei filtri.

Capitolo 3

Kathará

3.1 Introduzione

DEFINIZIONE Kathará

Kathará è un framework open source basato su container Docker che permette di creare, configurare ed eseguire reti virtuali, integrando in un unico ambiente tecnologie come **SDN** (Software Defined Network) e **NFV** (Network Function Virtualization) [7].

Ogni nodo di rete è simulato con un container. Il comportamento del nodo dipende dall'immagine usata, permettendo l'uso di protocolli tradizionali (OSPF, BGP, ecc), SDN o servizi applicativi (DNS, Apache, ecc). I vari dispositivi comunicano tra loro attraverso LAN virtuali di livello 2, riproducendo il comportamento di un collegamento fisico.

La scelta di basarsi su container invece che su macchine virtuali, rende Kathará uno strumento molto leggero. Ogni container richiede risorse nettamente inferiori rispetto a una macchina virtuale tradizionale, permettendo di simulare topologie complesse su hardware meno performante. Inoltre, permette di sfruttare bene tecnologie come NFV (Network Function Virtualization), che utilizza funzioni software per sostituire dispositivi hardware, e SDN (Software Defined Network), che permette di separare la logica di controllo della rete, dalla logica di invio dei pacchetti.

3.2 Architettura di Kathará

L'architettura di Kathará è composta da tre moduli principali: **Scripts**, **Operations** e il **Container Platform Adapter** [7].

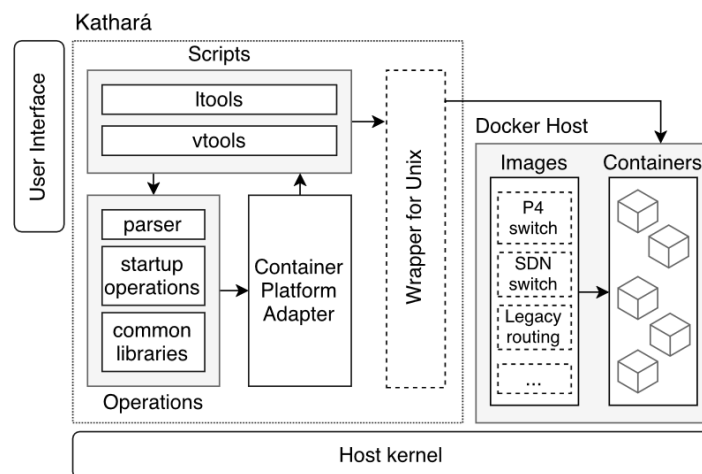


Figura 3.1: Architettura di Kathará [7]

Il modulo Scripts contiene le operazioni di base che utente può fare per avviare, fermare e mettere in pausa i nodi di rete. Si dividono in due tipologie:

1. **ltools:** valgono per tutti i nodi inseriti nel file di configurazione (**lstart**, **lclean**).
2. **vtools:** valgono per un singolo nodo (**vstart**, **vclean**).

Il modulo Operations, invece, racchiude alcune funzionalità che servono a Kathará per funzionare, come:

- **Parser:** traduce i file di configurazione o i parametri passati da riga di comando in nodi della rete con eventuali collegamenti. Si occupa anche di tradurre ulteriori opzioni, come risorse assegnate a ogni nodo o immagini Docker da utilizzare.
- **Start-Up Operations:** serve a garantire compatibilità fra i diversi sistemi.
- **Common Libraries:** genera una serie di operazioni indipendenti da eseguire.

Infine, il modulo del Container Platform Adapter converte i comandi generati in precedenza in direttive Docker (o altre piattaforme) per creare e collegare fra di loro i container. L'output dell'adapter ritorna al modulo Scripts, che lo invia al Docker daemon.

Nei sistemi Unix, per ragioni di sicurezza, i comandi devono prima essere approvati da un wrapper che scarta qualsiasi comando non ritenuto sicuro.

3.3 Configurazione

Supponiamo di avere un rete con tre nodi: due host e uno switch. Tramite Kathará è possibile crearli e avviarli tramite riga di comando, con i vtools, oppure, tramite file di configurazione, con gli ltools [7]. Questa seconda modalità è quella che userò per configurare la rete.

Il primo file da creare è quello in cui vengono definiti i nodi e se necessario altre opzioni, come le immagini dei container Docker.

KATHARÁ lab.conf

```
# Collegamento fra un host e porte dello switch
pc1[0]="A"
sw1[0]="A"

pc2[0]="B"
sw1[1]="B"

# Immagine per lo switch
sw1[image]="kathara/quagga"
```

Successivamente, è possibile creare e avviare i container con il comando **kathara lstart**, partendo dalle immagini di default o da quelle specificate. Kathará, inoltre, aprirà per ogni nodo un terminale, in modo da poter fare modifiche o test.

In alternativa, è possibile creare dei file di configurazione per ogni nodo, scrivendo al suo interno una lista di comandi da far fare in automatico all'accensione del container. Per l'host **pc1**, nel suo file di configurazione si può assegnare un ip e accendere la rispettiva interfaccia.

KATHARÁ pc1.startup

```
# Assegnare un ip
ip addr add 192.168.1.1/24 dev eth0

# Accendere l'interfaccia eth0
ip link set dev eth0 up
```

Tramite questa modalità è possibile configurare pure il secondo host e lo switch. Una volta digitato il comando **kathara lstart**, tutti i file di configurazione verranno usati per creare la rete.

3.3.1 Riproduzione dello scenario HTB in Kathará

Sfruttando Kathará, si possono riproporre gli esempi del capitolo precedente sull'utilizzo delle qdisc tramite il comando `tc`, testandolo con diversi nodi. In particolare, lo scenario su HTB è utile per dimostrare che è possibile fare traffic shaping con questo framework.

Facendo riferimento allo scenario dell'esercizio 2.3.2, il primo file da creare sarà `lab.conf`, in cui dovranno essere definiti i tre clienti e il server con i servizi.

KATHARÁ lab.conf

```
# Clienti
c1[0] = "A"
c2[0] = "A"
c3[0] = "A"

# Server
server[0] = "A"
```

Specificando per ognuno il dominio "A", Kathará simulerà il comportamento di una rete locale con uno switch che collega tutti i nodi. Successivamente, per ogni host, dovrà essere creato un file che conterrà tutti i comandi necessari per poter comunicare fra di loro e nel caso del server pure quelli utili per il traffic shaping.

KATHARÁ c1.startup

```
# Assegnazione ip
ip addr add 10.0.0.1/24 dev eth0

# Accensione interfaccia eth0
ip link set dev eth0 up
```

Questa configurazione sarà uguale anche per gli altri clienti, ad eccezione dell'indirizzo ip differente. Invece, in `server.startup`, verranno inseriti pure i comandi con `tc`.

BASH server.startup

```
# Assegnazione ip
ip addr add 10.0.0.10/24 dev eth0

# Accensione interfaccia eth0
ip link set dev eth0 up

# Nodo root e genitore
tc qdisc add dev eth0 root handle 1: htb default 30
tc class add dev eth0 parent 1: classid 1:1 htb rate 100mbit ceil 100mbit

# Classe sito web
tc class add dev eth0 parent 1:1 classid 1:10 htb rate 50mbit ceil 100mbit

# Classe servizio streaming
tc class add dev eth0 parent 1:1 classid 1:20 htb rate 35mbit ceil 60mbit

# Classe servizio file hosting
tc class add dev eth0 parent 1:1 classid 1:30 htb rate 15mbit ceil 20mbit
```

Infine, all'interno di quest'ultimo file, o sul container una volta avviato, dovranno essere creati pure i filtri per classificare i flussi.

BASH `server.startup`

```
# Filtri per sito web
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 80 0xffff
↪ flowid 1:10
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip sport 443
↪ 0xffff flowid 1:10

# Filtro per servizio streaming
tc filter add dev eth0 protocol ip parent 1:0 prio 2 u32 match ip sport 554
↪ 0xffff flowid 1:20

# Filtro per servizio file hosting
tc filter add dev eth0 protocol ip parent 1:0 prio 3 u32 match ip sport 21 0xffff
↪ flowid 1:30
```

Risultati

Per capire gli effetti che il traffic shaping ha portato alla rete, è utile confrontare i tempi di invio di dati con o senza HTB. La formula per calcolare i tempi di invio ideali è:

$$t = \frac{\text{dim} \times 8}{\text{rate}}$$

Sul nodo del server è stato creato un file di 100 MB e poi è stato posto in ascolto sulle tre porte: 443, 554 e 21. Ogni client ha scaricato contemporaneamente il file, tramite il comando `nc`, collegandosi alla porta del proprio servizio.

Per ciascun trasferimento è stato misurato il tempo reale.

Classe	Tempo Ideale	Tempo Reale
1:10	~ 16s	19.20s
1:20	~ 23s	23.33s
1:30	~ 53s	48.47s

Nell'esempio, si può notare come la classe 1:30 abbia impiegato meno tempo di quello ideale. Questo è dovuto al fatto che, dopo che il primo cliente ha finito di leggere i 100MB di file, le altre due classi si sono divise la banda tramite l'algoritmo di prestito.

Inoltre, per capire l'effetto del traffic shaping nella rete, è utile guardare anche i risultati senza l'utilizzo di HTB.

Classe	Tempo
Sito Web	22.81s
Streaming	23.04s
File Hosting	24.64s

Si può notare come i tempi, simili tra loro, rappresentino un terzo della banda totale, ossia circa 33Mbit/s per ciascun nodo, dato che $\frac{100\text{MB} \times 8}{33\text{Mbit/s}} = 24\text{s}$.

In conclusione, applicando una disciplina di accodamento come HTB per fare traffic shaping, ha permesso al servizio del sito web di avere a disposizione una banda maggiore, terminando il trasferimento per primo. Invece, ha rallentato il servizio di file hosting, dedicandogli una banda minore.

L'utilizzo di Kathará per creare la rete, ha permesso di calcolare i tempi reali in una LAN nelle diverse modalità proposte.

Servizio	Tempo con HTB	Tempo senza HTB
Sito Web	19.20s	22.81s
Streaming	23.33s	23.04s
File Hosting	48.47s	24.64s

3.4 Dipendenza dai moduli del kernel host

L'uso di container Docker rende la simulazione leggera e flessibile, permettendo di scegliere per ogni nodo un'immagine diversa, ma crea un limite importante. A differenza delle macchine virtuali, i container non eseguono un kernel proprio, ma condividono quello dell'host su cui gira Docker. Ciò che un nodo può fare a livello di rete dipende dai moduli caricati nel kernel dell'host.

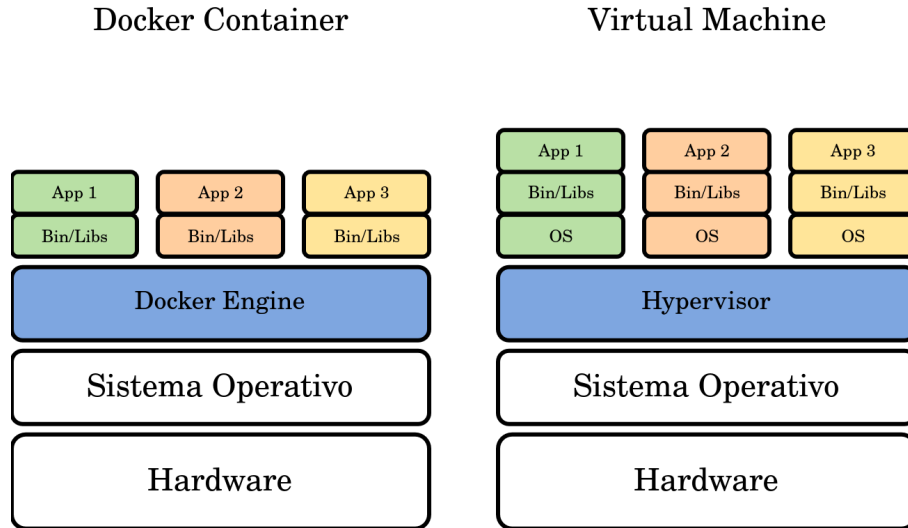


Figura 3.2: Docker Container vs Virtual Machine

Il comando `tc`, eseguito dentro un container, iniva solamente richieste al kernel, non implementa effettivamente le discipline di accodamento. Le `qdisc`, le classi e i filtri sono realizzati da moduli del kernel [1]:

Modulo	Descrizione
<code>sch_tbf</code>	Modulo per <code>qdisc</code> TBF
<code>sch_htb</code>	Modulo per <code>qdisc</code> HTB
<code>cls_u32</code>	Modulo per filtri u32

Per controllare la disponibilità, è possibile farlo tramite il comando `lsmod`. L'assenza di questi moduli porta a diverse conseguenze:

- **Errore nel container:** restituisce un errore al momento della creazione della `qdisc`.
- **Non riproducibilità:** lo stesso scenario può funzionare su un host e non su un altro;
- **Non portabilità:** sistemi operativi come Windows e MacOS richiedono l'uso di una virtual machine Linux [8] per usare Docker, che potrebbe non includere tutti i moduli necessari.

Una soluzione potrebbe essere spostare lo shaping del traffico in userspace, come verrà spiegato nel capitolo 4.

Capitolo 4

Traffic Shaping in Userspace

Come spiegato nel capitolo precedente, l'uso del comando `tc` in ambienti containerizzati come Kathará presenta limiti legati alla disponibilità di moduli all'interno del kernel dell'host. Per garantire portabilità e riproducibilità indipendentemente dal sistema di esecuzione è necessario cambiare approccio, spostando la logica di shaping all'interno dello userspace.

INTUIZIONE Prelevare i pacchetti dal Kernel

Una soluzione per spostare lo shaping in user space potrebbe essere quella selezionare correttamente e fermare momentaneamente i pacchetti nel kernel, permettendo a un processo in user space di decidere quanto e se trattenerli.

Questa modalità, però, richiede di intercettare specifici pacchetti, bloccarli per un determinato tempo e avere un processo capace di decidere quali scartare e quali rallentare.

4.1 Iptables

Il primo punto da chiarire è come selezionare i pacchetti corretti su cui effettuare lo shaping. Nel kernel Linux questo è possibile farlo tramite il framework **Netfilter**, che implementa lo strumento **Iptables** in user space [9].

DEFINIZIONE Iptables

Netfilter mette a disposizione una serie di **hooks**, in cui possono essere associate delle regole per definire i criteri per scartare, accettare o inoltrare un pacchetto. **iptables** è il comando in user space che permette di creare queste regole, raccogliendole in **catene**. Le catene, a loro volta, vengono inserite all'interno di specifiche **tabelle** [6].

Esistono diverse catene disponibili di base:

- **PREROUTING**: intercetta tutti i pacchetti in ingresso nell'interfaccia di rete, prima che venga effettuato il routing.
- **INPUT**: gestisce i pacchetti destinati ai processi in esecuzione sul'host.
- **FORWARD**: intercetta i pacchetti che l'host deve instradare verso un'altra destinazione.
- **OUTPUT**: gestisce i pacchetti che i processi in esecuzione sul'host devono inviare sulla rete.
- **POSTROUTING**: intercetta i pacchetti in uscita dopo che è stata effettuata la decisione di instradamento.

Ma anche diverse tabelle indipendentemente possibili:

- **Filter**: tabella di default. Usata per le regole di filtraggio sui pacchetti.
- **Nat**: tabella con le regole di NAT.

- **Mangle:** usata per modificare parti dei pacchetti.
- **Raw:** assegna un mark ai pacchetti, per non essere gestiti dal sistema tracciamento dello stato delle connessioni (**conntrack**). Questa tabella ha priorità più alta e viene chiamata prima delle altre.

La combinazione della tabella **Raw**, visto l'elevata priorità che gli viene assegnata, con la catena **OUTPUT** che permette di prendere i pacchetti prodotti da un processo in esecuzione nell'host, ci permette di risolvere la prima parte dell'intuizione di inizio capitolo 4: selezionare i pacchetti corretti per lo shaping, specificando, nel caso, porte, protocolli o altre caratteristiche con il comando **iptables**.

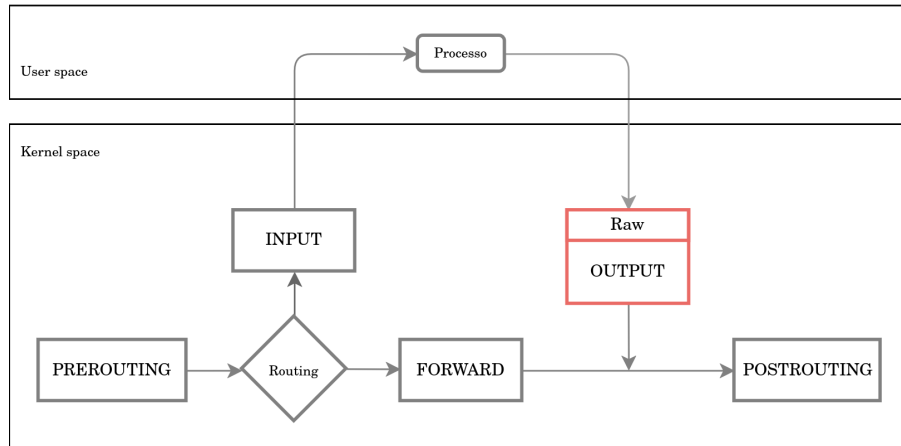


Figura 4.1: Rappresentazione della situazione descritta.

4.1.1 Target NFQUEUE

Avendo selezionato i pacchetti corretti, è necessario capire come poterli trattenere nel kernel, per metterli a disposizione del processo in user space.

La soluzione è nelle azioni o **target**, messe a disposizione da Netfilter [9], che normalmente vengono usate quando una regola trova corrispondenza:

1. **ACCEPT:** accetta il pacchetto.
2. **DROP:** scarta il pacchetto.
3. **RETURN:** interrompe la catena corrente.
4. **NFQUEUE:** accoda il pacchetto in una coda numerata nel kernel.

Il target **NFQUEUE** fa esattamente quello che serve nella soluzione proposta a inizio capitolo.

Nello specifico, si hanno diverse fasi:

1. Il pacchetto viene selezionato tramite il comando **iptables**, specificando i criteri che servono.
2. Il target **NFQUEUE** permette di spostarlo in una coda all'interno del kernel.
3. Il kernel, tramite **netlink** [10], crea un messaggio con i dati del pacchetto che servono al processo in userspace e lo invia tramite socket.
4. Il processo in userspace, tramite la libreria **libnetfilter_queue** [11], riceve il messaggio e decide come gestire il pacchetto, potendo anche ritardare la decisione, effettuando di conseguenza un'azione di shaping.

4.1.2 Netlink

DEFINIZIONE Netlink

Netlink è un protocollo utilizzato per far comunicare processi in kernel space con processi in userspace.

Mette a disposizione un'interfaccia socket che comunica con una API del kernel, permettendo quindi lo scambio di informazioni. Esistono diversi tipi di **famiglie**, che permettono di scegliere con che gruppo comunicare. Per le NFQUEUE esiste `NETLINK_QUEUE` [10].

4.1.3 Libnetfilter_queue

DEFINIZIONE libnetfilter_queue

Libreria in userspace che tramite una API permette di interagire con i pacchetti fermati in una coda nel kernel [11].

Sfrutta il protocollo Netlink per comunicare con il kernel, permette al processo in userspace di fare diverse azioni:

1. **Accept:** fa passare il pacchetto alle regole e catene successive.
2. **Drop:** fa scartare il pacchetto.
3. Alterare il contenuto del pacchetto.

Funzionando in maniera asincrona, può mandare la decisione in un qualsiasi momento, visto che richiede solo l'id del pacchetto. Questo è il dettaglio che permette al processo di effettuare lo shaping.

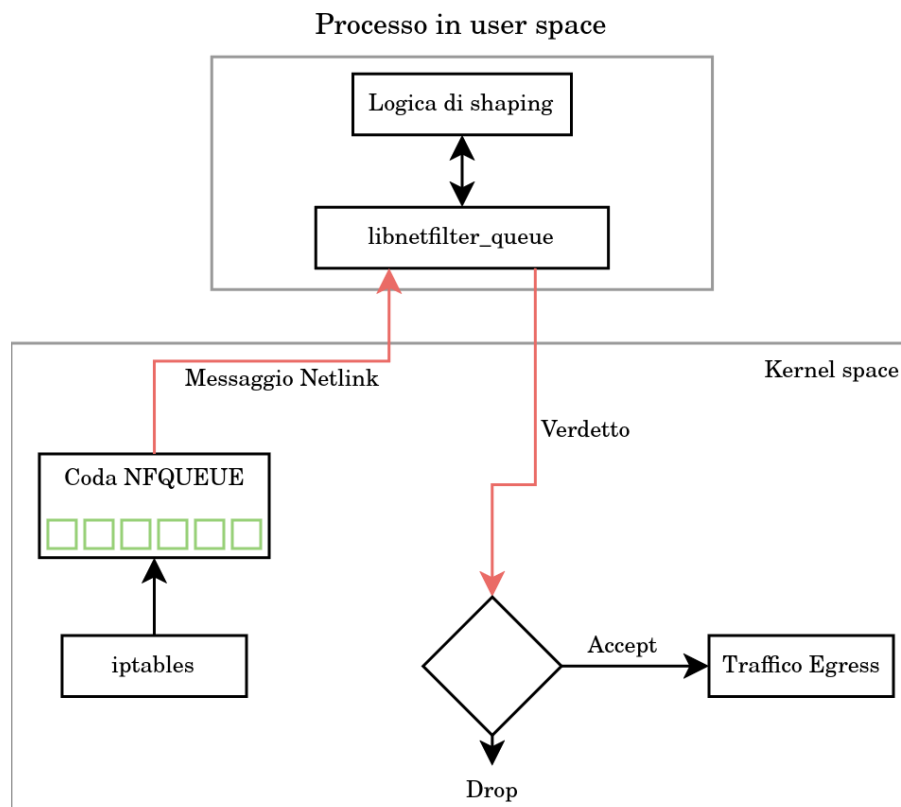


Figura 4.2: Ciclo di vita pacchetto NFQUEUE.

L'ultimo pezzo mancante è il programma in user space che implementi tutta la logica di shaping vista nel capitolo 2.

4.2 Damper

DEFINIZIONE Damper

Damper [12] è programma open-source scritto in linguaggio **C**, che tramite la libreria `<libnetfilter_queue.h>` messa a disposizione da Netfilter [9], permette di interagire con i pacchetti nella coda NFQUEUE nel kernel.

Implementa, tramite un file di configurazione, una logica di shaping simile a un algoritmo **leaky bucket**, dove però latenza dipende dalla dimensione del pacchetto.

DEFINIZIONE Leaky Bucket

A differenza dell'algoritmo token bucket, che accumola gettoni all'interno di un contenitore e invia pacchetti solo se si hanno abbastanza gettoni, l'algoritmo leaky bucket usa un approccio diverso: il bucket raccoglie tutti i pacchetti e li invia a una velocità costante, non vengono usati token.

4.2.1 Logica di funzionamento

La configurazione dei parametri per il traffic shaping viene fornita tramite il file `damper.conf`. All'interno del programma la funzione `read_config()` si occupa di fare il parsing del file e di salvare tutte le informazioni nella struttura dati `struct userdata`.

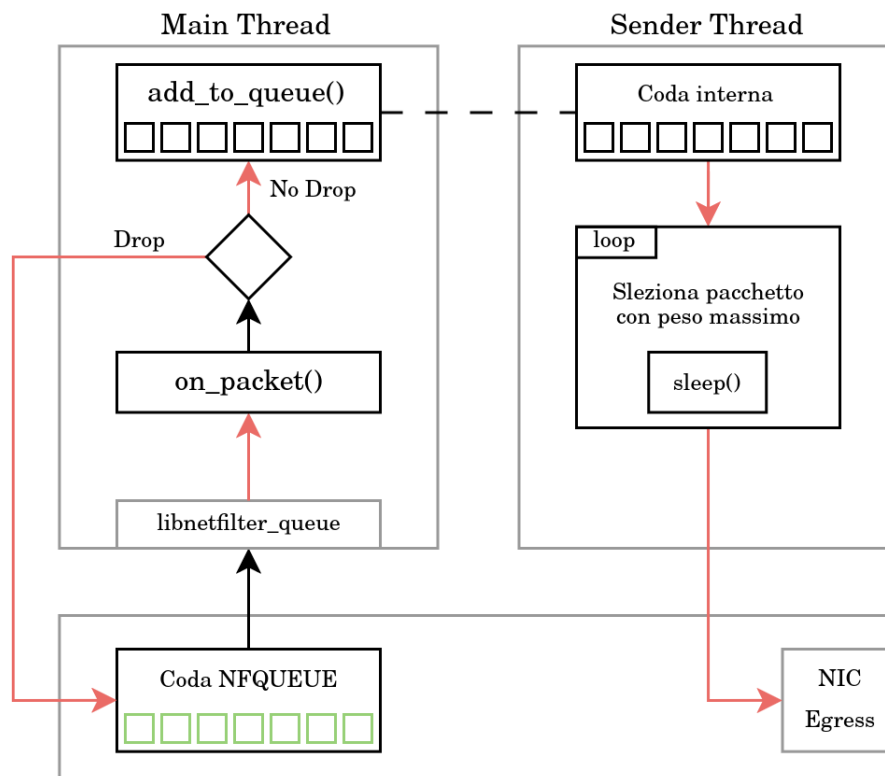


Figura 4.3: Funzionamento di Damper.

La logica principale è divisa in due thread: il **main thread** e il **sender thread**.

Il primo, è il thread che tramite le funzioni fornite dalla libreria `<libnetfilter_queue>` gestisce i pacchetti della NFQUEUE. Utilizza, oltre a `read_config()`, una serie di funzioni:

- `on_packet()`: funzione che gestisce ogni pacchetto della NFQUEUE, assegnandoli un **peso**. Se negativo, viene inviato un verdetto di Drop per scartare il pacchetto, altrimenti viene invocata un'altra funzione per inserire il pacchetto in una coda.

- `add_to_queue()`: funzione che copia l'id, la dimensione e il payload del pacchetto in un array di `struct packet`, ossia una struttura dati usata per contenere i dati di un pacchetto della NFQUEUE in user space. Se il vettore è pieno, viene inviato un verdetto di Drop per scartare il pacchetto con il peso minore.

Il secondo thread, invece, implementa lo shaping tramite la funzione `sender_thread()`. Ciclo continuo che, dall'array dei pacchetti, trova quello con il peso maggiore e genera un verdetto di Accept. Alla fine di ogni ciclo, però, viene fatta una sleep di un numero di secondi, per fare lo shaping effettivo. La formula per il calcolo del tempo è la seguente:

$$t = \frac{(\text{packet.size} * 1000000000)}{\text{rate}}$$

Il risultato saranno i nanosecondi che rallenteranno l'invio del verdetto.

4.2.2 Configurazione

Damper, per funzionare ha bisogno di un file di configurazione, ossia `damper.conf`. All'interno si possono configurare i diversi moduli per calcolare il peso dei pacchetti e inserire i parametri per la parte di traffic shaping.

DAMPER damper.conf

```
# nfqueue queue id
queue 3

# traffic limit in bits per second (suffixes K, M and G allowed)
limit 20M

# queue length
packets 100

# modules
# number of recent flows
inhibit_big_flows nrecent 10
# dump debug info every 5 seconds
#inhibit_big_flows debug 5

# entropy module
entropy nrecent 10
#entropy debug 5
entropy k 2.0
# you can override source or destination port for this module
#entropy sport 0
#entropy dport 0

#bymark module
#traffic with mark 101 gets weight 0.2
bymark 101 0.2
#traffic with mark 102 gets weight 0.5
bymark 102 0.5
```

Per usarlo è possibile clonare la repository disponibile su GitHub [12]:

BASH Clonazione

```
git clone https://github.com/vmxdev/damper.git
```

Successivamente è possibile compilarlo, installando prima l'unica dipendenza del programma: `libnetfilter_queue`.

BASH Compilazione

```
gcc -Wall -pedantic damper.c modules.conf.c -o damper -lnetfilter_queue -pthread  
↪ -lrt -lm
```

Dopo aver ottenuto l'eseguibile **damper**, è possibile avviarlo, passando come unico parametro il file di configurazione.

BASH Esecuzione

```
./damper damper.conf
```

Nel prossimo capitolo 5, dove aver trovato una modalità per fare traffic shaping in user space, si dimostrerà la fattibilità e possibili limiti.

Capitolo 5

Casi di Studio

5.1 Introduzione

Nel capitolo 2 è stato dimostrato il funzionamento del traffic shaping con il comando `tc`, citando anche i limiti che gli appartengono. Invece, in questo capitolo, si vuole dimostrare la fattibilità del traffic shaping tramite l'utilizzo di Kathará e Damper, elencando eventuali limiti presenti. Saranno tre diversi scenari, ognuno utile a capire i diversi aspetti della strategia di shaping in userspace.

Per usare correttamente Damper tramite Kathará, ho creato un'immagine Docker, al cui interno ho compilato il programma come visto nel capitolo 4 e aggiunto un file `damper.conf` come base in `/etc/damper/`.

DOCKERFILE Dockerfile personalizzato

```
# Usata la stessa immagine di base per compilare
FROM kathara/core:latest AS damper
RUN apt update && apt install -y --no-install-recommends \
    libnetfilter-queue-dev \
    gcc \
    build-essential && \
    apt clean && rm -rf /tmp/* /var/lib/apt/lists/* /var/tmp/*
WORKDIR /src
RUN git clone --depth 1 https://github.com/vmxdev/damper.git .
RUN gcc -Wall -pedantic damper.c modules.conf.c -o damper -lnetfilter_queue
↳ -pthread -lrt -lm

# Immagine kathara/base
FROM kathara/core:latest
LABEL org.opencontainers.image.authors="Kathara Team <contact@kathara.org>"
RUN apt update && apt upgrade -y && apt install -y --no-install-recommends \
    libnetfilter-queue-dev \
    apache2 \
    apache2-bin \
    bind9 \
    bind9-utils \
    dnsmasq && \
    apt clean && rm -rf /tmp/* /var/lib/apt/lists/* /var/tmp/*
RUN ln -s /etc/init.d/named /etc/init.d/bind
COPY apache2.service /usr/lib/systemd/system/apache2.service

RUN mkdir /etc/damper
RUN mkdir -p /var/lib/damper
COPY --from=damper /src/damper /usr/local/bin/damper
COPY --from=damper /src/damper.conf /etc/damper/damper.conf

#Opzionale uno dei due o altra versione
#RUN echo "alias damper='damper /etc/damper/damper.conf &'" >> /root/.bashrc
RUN echo "alias shaper='damper /etc/damper/damper.conf &'" >> /root/.bashrc
```

5.2 Limitazione tra due Nodi

Il primo scenario è molto semplice e serve per dimostrare un caso di limitazione di banda tra due nodi nella stessa rete locale.

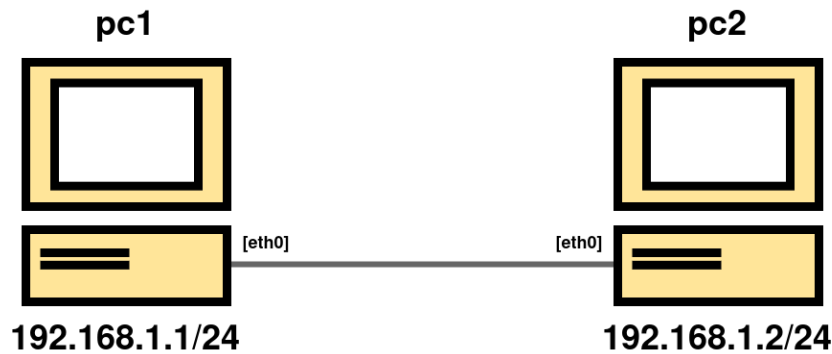


Figura 5.1: Primo scenario.

Sono presenti due nodi host, **pc1** e **pc2**, il cui traffico è limitato in entrambi secondo questi parametri:

- **Rate:** 1 Mbit/s.
- **Burst:** 10 KB.
- **Latency:** 50 ms.

5.2.1 Soluzione

In questo primo esercizio, verrà mostrata pure la configurazione per Kathará. Nei prossimi non sarà presente, dato che sarebbe stata molto simile e quindi una ripetizione inutile. Nello specifico, il primo file è `lab.conf`, in cui si definisce la struttura principale della rete.

KATHARÁ `lab.conf`

```
# Nodo pc1
pc1[0]="A"
pc1[image]="kathara/damper"

# Nodo pc2
pc2[0]="A"
pc2[image]="kathara/damper"
```

Ovviamente, l'immagine usata per i due nodi sarà quella personalizzata con Damper.

I due host `pc1` e `pc2` a loro volta dovranno avere il loro file di startup, che segue questo modello:

KATHARÁ Modello base per file `.startup`

```
#<host>.startup
ip addr add 192.168.1.x/24 dev eth0
ip link set eth0 up
```

Una volta avviata la rete con il comando `kathara lstart`, sarà possibile configurare il traffic shaping in user space. Per prima cosa, dato che non sono presenti vincoli, si deve intercettare tutto il traffico nella tabella OUTPUT per inserirlo in una NFQUEUE nel kernel:

BASH Regola iptables

```
iptables -t raw -A OUTPUT -j NFQUEUE --queue-num 0 --queue-bypass
```

Successivamente, andranno modificati i parametri nel file `damper.conf`, per rispettare le specifiche richieste nello scenario:

DAMPER damper.conf

```
# nfqueue queue id
queue 0

# traffic limit in bits per second (suffixes K and M allowed)
limit 1M

# queue length
packets 7
```

Così facendo, il rate è limitato a 1Mbit/s e il burst di 10 KB, usando un MTU di 1500 byte, corrisponde a circa 7 pacchetti. Con questi valori si hanno circa 50 ms di latenza.

Infine, serve avviare Damper con il comando: `damper /etc/damper/damper.conf`.

5.2.2 Risultati

Per verificare il funzionamento, si può provare a trasferire un file da un host all'altro, come esempio possiamo scegliere di valutare i tempi da pc1 a pc2. Per far ciò, è necessario creare un file da poter spedire nel nodo pc1:

BASH File da spedire da 1MB

```
dd if=/dev/urandom of=file.bin bs=1M count=1
```

Successivamente, si mette il nodo pc2 in ascolto con il comando:

BASH Comando Netcat

```
nc -l -p 8080 > /dev/null
```

Infine, si invia il file dal nodo pc1, monitorando il tempo richiesto:

BASH Invio di file.bin

```
time sh -c "cat file.bin | nc 192.168.1.2 8080 -q1"
```

Si può notare che il file, per essere spedito dall'host pc1 all'host pc2, ha impiegato 10 secondi, rispettando quindi il rate di 1Mbit/s.

Il tempo ideali di trasferimento, facendo riferimento alla formula nel capitolo 2, sarebbe:

$$t = \frac{\text{dim} \times 8}{\text{rate}} = \frac{1 \text{ MB} \times 8}{1 \text{ Mbit/s}} = 8s$$

Un margine di circa 2s rimane del tutto accettabile, visto la complessità aggiunta dal dover lavorare in user space.

5.3 Limitazione Differenziata verso più Nodi

Con il secondo scenario, si vuole dimostrare che con Damper è possibile riproporre un funzionamento simile alla disciplina di coda con classi HTB 2.3.2. Non sarà, quindi, limitato tutto il traffico in uscita, ma valutando la coppia di nodi che comunica.

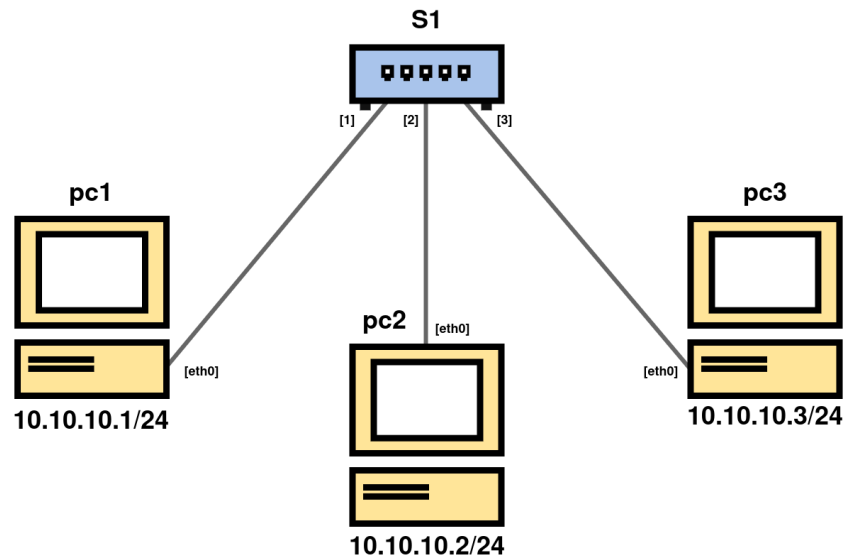


Figura 5.2: Secondo scenario.

In questo esempio sono presenti tre nodi: **pc1**, **pc2** e **pc3**, tutti nella stessa rete locale. Il traffico è diviso in due flussi con velocità diverse:

1. **Slow:** Banda = 1 Mbit/s.
2. **Fast:** banda ≥ 50 Mbit/s o non limitata.

Ogni coppia di nodi ha diverse combinazioni, per esempio:

	pc1	pc2	pc3
pc1		slow	slow
pc2	slow		fast
pc3	slow	fast	

5.3.1 Soluzione

In questo caso, se prendiamo come riferimento l'host pc1, dovrà limitare il traffico sia verso pc2 che verso pc3.

Damper, però, non simula il comportamento di htb e non supporta nemmeno la definizioni di classi, quindi per farlo funzionare in questo esercizio, sarà necessario creare due processi diversi, ognuno con il proprio file di configurazione. I due file avranno i seguenti parametri:

DAMPER /etc/damper/damper-pc2.conf

```
# nfqueue queue id
queue 0

# traffic limit in bits per second (suffixes K and M allowed)
limit 1M

# queue length
packets 7
```

DAMPER `/etc/damper/damper-pc3.conf`

```
# nfqueue queue id
queue 1

# traffic limit in bits per second (suffixes K and M allowed)
limit 1M

# queue length
packets 7
```

Inoltre, il traffico dovrà essere inserito in due NFQUEUE diverse, necessitando di due diverse regole con `iptables`, dove viene specificato l'ip di destinazione e l'id della coda:

BASH Regola traffico verso pc2

```
iptables -t raw -A OUTPUT -d 10.10.10.2 -j NFQUEUE --queue-num 0 --queue-bypass
```

BASH Regola traffico verso pc3

```
iptables -t raw -A OUTPUT -d 10.10.10.3 -j NFQUEUE --queue-num 1 --queue-bypass
```

Osservando questo esempio, si può notare che i due flussi hanno lo stesso valore di rate, portando a pensare di poter usare lo stesso file e la stessa istanza di Damper per entrambi.

Questa soluzione, però, è errata e la risposta si trova nel funzionamento del programma. Se i due flussi condividessero lo stesso processo, i loro pacchetti sarebbero inseriti nella stessa NFQUEUE in kernel space e nello stesso array in userspace, creando due principali problemi:

1. Se le priorità dei pacchetti del primo flusso fossero più elevate rispetto a quelle del secondo e nel caso l'array si riempisse, i pacchetti del secondo flusso verrebbero scartati in favore di quelli del primo.
2. Il valore di 1 Mbit/s non sarebbe garantito a entrambi, ma sarebbe diviso fra i due flussi.

Per finire e valutare i risultati, devono essere avviate le due istanze del programma:

BASH

```
damper /etc/damper/damper-pc2 &
damper /etc/damper/damper-pc3 &
```

5.3.2 Risultati

Per valutare le prestazioni si possono ripetere le modalità della sezione precedente 5.2.2, tenendo in considerazione entrambi i flussi verso i rispettivi host.

Inizialmente, dovranno essere messi in ascolto i due nodi pc1 e pc2:

BASH Server nc

```
# pc2
nc -l -p 8080 > /dev/null

# pc3
nc -l -p 8080 > /dev/null
```

Per poter calcolare il tempo impiegato dal nodo `pc1` per inviare a entrambi il `file.bin`, creato in modo uguale alla sezione 5.2.2.

BASH Nodo pc1

```
time sh -c "cat file.bin | nc 10.10.10.2 8080 -q1" & time sh -c "cat file.bin | nc
↪ 10.10.10.3 8081 -q1" &
```

BASH Risultato

```
real      0m10.054s
user      0m0.003s
sys       0m0.007s

real      0m10.151s
user      0m0.003s
sys       0m0.007s
```

Il tempo di entrambi i flussi, 10 secondi ciascuno, è uguale a quello ottenuto nella sezione 5.2.2, rimanendo quindi fedeli al tempo ideale.

Inoltre, grazie a questo scenario, si è messo un po' di luce su un limite su `htb` (e in parte `tbpf`) di questo programma in user space. Sarà la principale motivazione del lavoro fatto nel capitolo 6.

5.4 Limitazione Banda con Gateway/Router

Quest'ultimo esempio presenta uno scenario diverso rispetto agli altri. Si vuole limitare il traffico verso un host della rete da una sorgente di cui non si ha controllo, quindi su cui non si può applicare il traffic shaping con `Damper`.

Nella realtà, è una situazione molto comune e di solito viene gestita attraverso router o gateway.

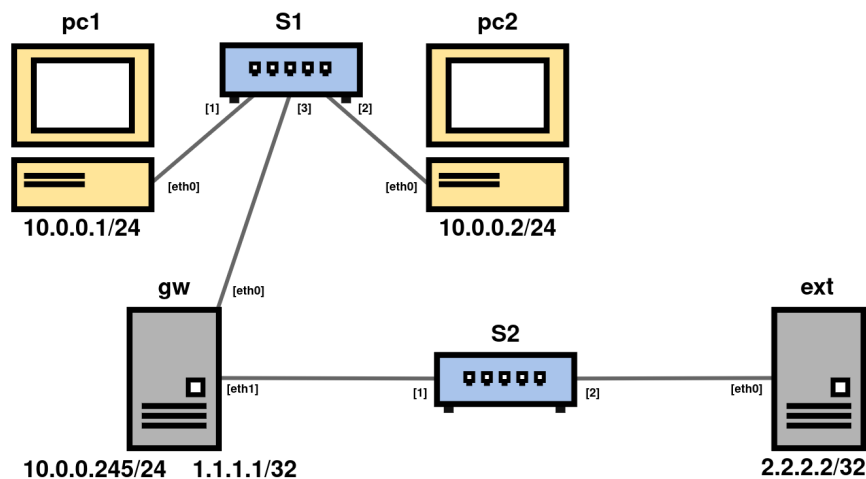


Figura 5.3: Terzo scenario.

Nelle reti locali sono presenti due host: `pc1` e `pc2` e un gateway `gw`, che si occuperà del traffic shaping. Quest'ultimo, dovrà pure permettere il routing fra i nodi della LAN con il server esterno e viceversa.

Si vuole limitare la banda di `pc1` per operazioni di download, per comodità i parametri saranno gli stessi degli esempi precedenti:

- **Rate:** 1 Mbit/s.
- **Burst:** 10 KB.
- **Latency:** 50 ms.

5.4.1 Soluzione

Come specificato nell'introduzione, lo shaping verrà effettuato dal nodo gw. Per iniziare, è necessario scrivere la regola con `iptables`, per intercettare tutto il traffico verso pc1, assegnandolo alla NFQUEUE numero 0:

BASH Regola iptables

```
iptables -t raw -A FORWARD -d 10.0.0.1 -j NFQUEUE --queue-num 0 --queue-bypass
```

In questo caso, non viene più usata la tabella OUTPUT, ma la tabella FORWARD, visto che i pacchetti non sono generati da un processo dell'host gw, ma da uno del server ext. Successivamente, come già visto più volte, va configurato il file `damper.conf` con i parametri giusti:

DAMPER damper.conf

```
# nfqueue queue id
queue 0

# traffic limit in bits per second (suffixes K and M allowed)
limit 1M

# queue length
packets 7
```

Infine, avviandolo, sarà possibile valutare i risultati.

BASH Istanza damper

```
damper /etc/damper/damper.conf
```

5.4.2 Risultati

Anche in questo scenario, verranno valutati i tempi di invio del file a pc1 tramite gw, paragonandolo, per completezza, al tempo ideale a 1 Mbit/s e al tempo di invio verso pc2. Nello specifico, dovranno essere messi in ascolto entrambi i client della rete locale:

BASH Server nc

```
# pc1
nc -l -p 8080 > /dev/null

# pc2
nc -l -p 8080 > /dev/null
```

E dal server ext, di cui in uno scenario reale non si ha controllo, inviamo `file.bin`, di dimensione 1 MB, verso gli host:

BASH Invio file.bin verso pc1 e pc2

```
time sh -c "cat file.bin | nc 10.0.0.1 8080 -q1" & time sh -c "cat file.bin | nc
↪ 10.0.0.2 8081 -q1" &
```

BASH Risultato

```
real      0m1.201s
user      0m0.002s
sys       0m0.019s

real      0m10.802s
user      0m0.213s
sys       0m0.607s
```

I tempi rispecchiano i risultati previsti, notando che verso pc1 siano stati impiegati 10 secondi e invece, verso pc2 sia stato impiegato un decimo del tempo.

5.5 Limiti di Damper

Nonostante Damper permetta di realizzare il traffic shaping in user space attraverso NFQUEUE, la sua architettura presenta alcune limitazioni quando si vuole gestire contemporaneamente traffico appartenente a classi diverse. Nello specifico, lo shaping è basato su un'unico vettore di pacchetti e su un unico limite di banda associato al processo in esecuzione.

Nel capitolo successivo, spiego il motivo che mi ha portato allo sviluppo di **Donper** [13], un fork di Damper nel quale ho introdotto una struttura gerarchica per la gestione dei pacchetti, implementando un l'algoritmo di prestito e la divisione in classi. Con questa modifica, è stata aggiunta una gestione dei pacchetti di Damper simile a discipline di accodamento come tbf e htb.

Capitolo 6

Donper

6.1 Introduzione

Donper [13] è un fork di **Damper** [12], che ne estende le funzionalità, implementando un funzionamento simile a HTB, seppur non uguale.

Nello specifico, è stata cambiata la struttura dati utilizzata dal precedente programma, per creare un albero **gerarchico** composta da un nodo padre con un numero variabili di nodi figli, ognuno con una coda prioritaria di pacchetti e i parametri necessari.

Inoltre, è stata implementato pure un algoritmo di **prestito** di banda, simile a quello usato nella disciplina HTB.

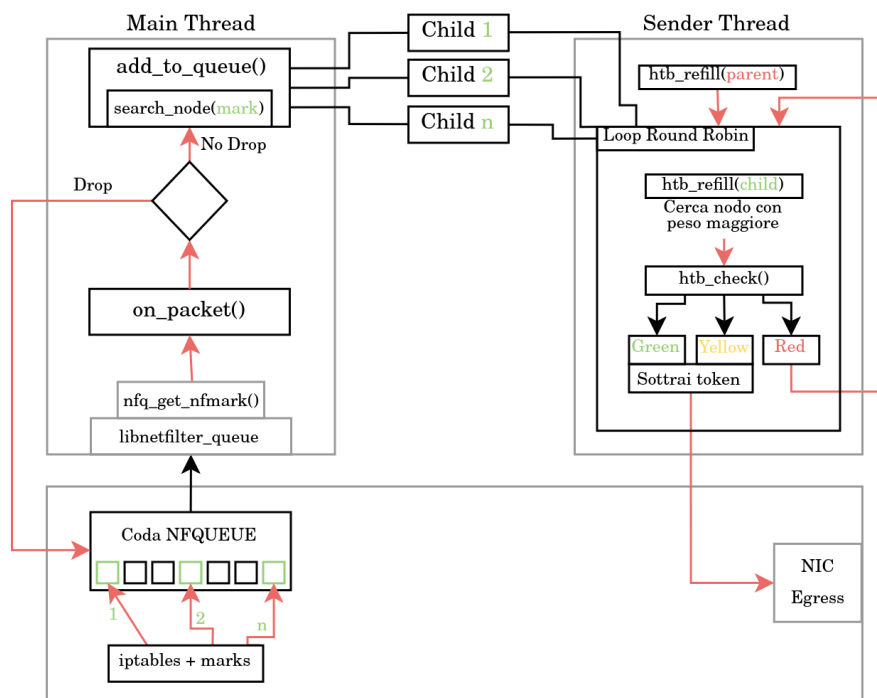


Figura 6.1: Architettura generale Donper.

Il programma modificato, come quello originale, mantiene due thread:

1. **main thread:** si occupa del parsing del file di configurazione, della creazione dei nodi dell'albero e della gestione e classificazione dei pacchetti.
2. **sender thread:** si occupa dell'invio dei pacchetti, utilizzando l'algoritmo token bucket e se necessario gestisce i prestiti fra nodi.

Nelle prossime sezioni, verrà descritta l'implementazione in modo dettagliato, dimostrando fattibilità e limiti di questa nuova soluzione.

6.2 Configurazione dei Parametri

La configurazione dei parametri di ogni classe è possibile tramite il file `donper.conf`:

```
DONPER donper.conf

# nfqueue queue id
queue 3

# traffic limit in bits per second (suffixes K, M and G allowed) for parent node
limit 40M
# bucket size for parent node in bytes
burst 50K

# htb
htb yes

# htb classes, limit >= sum(classes rates)
class mark 1 rate 20m ceil 40m burst 20k
class mark 2 rate 20m ceil 40m burst 20k
#class mark 3 rate 5m  ceil 10m burst 10k
#class mark 4 rate 5m  ceil 10m burst 10k

# modules
# number of recent flows
inhibit_big_flows nrecent 10
# dump debug info every 5 seconds
#inhibit_big_flows debug 5

# entropy module
entropy nrecent 10
#entropy debug 5
entropy k 2.0
# you can override source or destination port for this module
#entropy sport 0
#entropy dport 0
```

Dall'implementazione originale, è stata aggiunta la possibilità di decidere se usare le classi e il prestito con il parametro `htb`. Inoltre, è possibile configurare ogni classe con i suoi parametri principali:

- **Class Mark:** usato per definire la classe, il mark è possibile inserirlo tramite il comando `iptables`.
- **Rate:** velocità di generazione dei token garantita.
- **Ceil:** velocità massima possibile di generazione dei token.
- **Burst:** capienza del bucket della classe.

Anche la configurazione del nodo padre richiede due parametri: **limit** (rate) e **burst**.

La parte riguardante i moduli di assegnamento delle priorità ai pacchetti nell'array in user space, è stata mantenuta uguale. Rimuovendo, però, la possibilità di usare i mark per assegnare i pesi.

6.2.1 Struttura Dati

Il cambio della struttura del file di configurazione, ha richiesto due grandi cambiamenti. Il primo è la modifica della funzione `read_config()`, per fare il parsing. Il secondo, invece, è la creazione di due nuove strutture dati per la gestione dei parametri:

1. **Struct htb_parent:** utilizzata per gestire i parametri del nodo padre.
2. **Struct htb_child:** rappresenta ogni nodo figlio, gestisce i parametri e l'array di pacchetti.

C Struct htb_parent

```

struct htb_parent {
    int queue;           /* nfequeue queue id */
    struct nfq_q_handle *qh; /* queue handle */
    int nfqlen;          /* internal queue length */

    uint64_t limit;      /* or rate, bit/s */
    int64_t tokens;      /* token = byte */
    int64_t burst;       /* bucket size */

    struct timespec old_time; /* old time to generate new tokens */

    pthread_t sender_tid;
    pthread_mutex_t lock;

    int htb;             /* 1 for htb, 0 for tbf */
    struct htb_child **children; /* array of the children */
    size_t n_children;    /* number of children node */
};

```

All'interno, oltre i valori per lo shaping già elencati, sono contenuti pure:

- I valori per gestire la NFQUEUE tramite `libnetfilter_queue`, come l'id, l'handle e la lunghezza della coda.
- Tempo ultimo di refill dei token, usato per capire quanto tempo è passato dall'ultima volta che sono stati aggiunti gettoni.
- Pid del sender thread e mutex per gestire le risorse in modo sicuro.
- Array di nodi figli, con relativa lunghezza.

Invece, nella struttura dati di ogni nodo figlio:

C Struct htb_child

```

struct htb_child {
    uint32_t mark;

    uint64_t rate;        /* min, byte/s */
    uint64_t ceil;        /* max with borrowing, byte/s */

    int64_t tokens;       /* number of token in the bucket */
    int64_t burst;        /* bucket size */
    int64_t ceil_tokens;  /* number of token for borrowing */
    int64_t ceil_burst;   /* burst for borrowing */

    struct timespec old_time; /* old time to generate new tokens */
    struct timespec ceil_old_time;

    /* priority queue */
    struct mpacket *packets;
    double *prioarray;
    size_t qlen;
};

```

Oltre a tutti i valori per lo shaping è la divisione in classi, viene inserito pure un array di pacchetti, con i relativi pesi e la lunghezza.

Utile notare che in questo caso, a differenza del padre, servono due tempi diversi per riempire il bucket, dato che si può riempire a due velocità diverse. Infatti, deve essere gestito sia il caso in cui non ci sia il prestito, con `rate`, `burst` e `tokens`, che il caso in cui ci sia, con `ceil`, `ceil_burst` e `ceil_tokens`.

6.3 Classificazione dei Pacchetti

Nello scorso capitolo, è stato accennato come per classificare i pacchetti nelle varie code, venga usato il valore `mark`.

Netfilter [9], tramite il comando `iptables`, permette di inserire un valore da 32 bit nei pacchetti di rete nel kernel space. Questo permette di poter classificare i pacchetti tramite il numero assegnato, simulando il comportamento del parametro `class id` nella disciplina di coda HTB. In

Donper, chi si occupa di questa situazione è la funzione `on_packet()`:

C Parte della funzione `on_packet()`

```
mark = nfq_get_nfmark(nfad);

pthread_mutex_lock(&parent->lock);
if (weight < 0) {
    /* drop packet with with negative weight */
    nfq_set_verdict(parent->qh, id, NF_DROP, 0, NULL);
} else {
    /* add to queue with positive weight */
    add_to_queue(parent, p, id, plen, weight, mark);
}
pthread_mutex_unlock(&parent->lock);
```

Nello specifico, tramite la funzione `nfq_get_nfmark()`, salva il valore del `mark` del pacchetto e dopo aver controllato la positività del peso, chiama la funzione `add_to_queue()`. Queste operazioni vengono eseguite in modo sicuro tramite il `pthread_mutex`, per evitare una race condition fra thread.

Successivamente, all'interno della funzione `add_to_queue()`, la variabile `mark`, verrà usata per cercare la classe corretta fra i vari nodi figli, tramite la funzione `search_node()`:

C Funzione `search_node()`

```
static int
search_node(struct htb_parent *parent, uint32_t mark)
{
    int i;

    if (!parent->htb) {
        return 0;
    }

    for (i=0; i<parent->n_children; i++) {
        if (parent->children[i]->mark == mark)
            return i;
    }

    return -1;
}
```

Nel caso, sia stato scelto nel file di configurazione di non voler usare le classi, il valore di ritorno sarà sempre 0.

Il funzionamento principale della funzione `add_to_queue()`, avendo il nodo corretto, è pressochè uguale a Damper. Si occuperà, quindi, di aggiungere il pacchetto all'array, sostituendolo, se necessario, con quello con peso minore.

6.4 Gestione della Trasmissione di Pacchetti

Le funzioni delle sezioni precedenti erano gestite nel **thread principale**, invece, l'invio dei pacchetti, con il relativo shaping, deve essere fatto nel **sender thread**.

Inoltre, a differenza di Damper, il fork Donper deve essere in grado di gestire le diverse modalità, ossia con classi e prestito o senza.

6.4.1 Prestito fra classi

Per gestire il prestito fra classi, è necessario evidenziare le tre situazioni possibili in cui un nodo si può trovare:

1. **GREEN:** il numero di tokens è sufficiente per inviare immediatamente il pacchetto.
2. **YELLOW:** il numero di tokens non è sufficiente per inviare immediatamente un pacchetto, ma è possibile chiedere un prestito alla classe genitore, visto che la banda non è satura.
3. **RED:** il numero di tokens non è sufficiente per inviare immediatamente un pacchetto e non è possibile prendere in prestito ulteriore banda. Il pacchetto rimane in coda finché non sarà possibile spedirlo.

Nel programma questi tre casi sono descritti da:

```
C enum htb_mode
```

```
enum htb_mode
{
    RED = 0,
    GREEN,
    YELLOW
};
```

E gestiti dalla funzione `htb_check()`, che controllando il numero di tokens disponibili della classe, ritorna uno di questi tre valori:

```
C htb_check()
```

```
static enum htb_mode
htb_check(struct htb_parent *parent, struct htb_child *child, int64_t size)
{
    if ((child->tokens >= size)) {
        return GREEN;
    }

    if ((parent->limit != UINT64_MAX) && (parent->tokens < size)) {
        return RED;
    }

    if (child->ceil_tokens >= size) {
        return YELLOW;
    }

    return RED;
}
```

Invece, la condivisione di banda fra nodi figli è gestita automaticamente dalla logica del sender thread:

C Parte della funzione `sender_thread()`

```
mode = htb_check(parent, child, size);
if (mode == RED) {
    /* no send */
    continue;
}

vres = nfq_set_verdict(parent->qh, child->packets[idx].id, NF_ACCEPT, 0, NULL);
if (vres < 0) {
    fprintf(stderr, "nfq_set_verdict() failed, %s\n", strerror(errno));
}

child->tokens -= size;
parent->tokens -= size;
if (mode == YELLOW) {
    child->ceil_tokens -= size;
}
```

Nello specifico, per ogni classe, viene controllato il valore ritornato da `htb_check()`. Nel caso sia **RED**, viene usato l'istruzione `continue` per andare al nodo successivo, visto che non è possibile inviare. Altrimenti, viene inviato alla NFQUEUE nel kernel il verdetto `NF_ACCEPT`, tramite la funzione `nfq_set_verdict()`. Per ogni pacchetto, deve essere sempre essere rimossa la dimensione del pacchetto da il valore `tokens` del nodo padre e del nodo figlio attuale, invece, se il valore di ritorno è **YELLOW**, deve essere, pure, rimossa al valore `ceil_tokens` del nodo figlio corrente. I tokens di ogni nodo devono essere, però, generati secondo il loro valore di `rate/ceil`.

Per questo, è stata creata una funzione `htb_refill()`, che funzioni sia per il nodo padre che per tutti i nodi figli:

C Parte della funzione `htb_refill()`

```
clock_gettime(CLOCK_MONOTONIC, &now);

/* time passed in nano second */
ns = (now.tv_sec - old_time->tv_sec) * (int64_t)BILLION + (now.tv_nsec -
↳ old_time->tv_nsec);
if (ns <= 0) {
    return;
}

/* token generated */
add = (ns / (int64_t)BILLION) * (int64_t)rate + ((ns % (int64_t)BILLION) *
↳ (int64_t)rate) / (int64_t)BILLION;

*old_time = now;
```

I token vengono generati usando i parametri `old_time` `rate/ceil` del nodo passato. Per prima cosa, viene calcolato il tempo passato dall'ultima volta che è stata chiamata la funzione `htb_refill()`, successivamente viene trovato il valore corretto di tokens da aggiungere, tramite la formula:

$$n_tokens = time_passed \times rate/ceil$$

Nel codice, non potendo usare valori decimali, è stata usata una formula per aggiungere pure il resto corretto, visto che il casting a un intero avrebbe fatto perdere precisione. Infine, viene aggiornata la variabile `old_time`, per il prossimo uso.

6.4.2 Gestione senza Classi

Una modalità di utilizzo di Donper prevede il funzionamento anche senza classi, simulando la disciplina di coda TBF, visto che nel programma precedente Damper era implementato in modo diverso.

Tramite il file di configurazione `donper.conf` è possibile decidere con il parametro `htb` di voler utilizzare questa modalità.

In questo caso, nell funzione di inizializzazione delle strutture dati `tree_init()`:

C Parte di `tree_init()`

```
/* no class or no htb, then tbf */
if (parent->n_children == 0) {
    parent->n_children += 1;

    /* using one child node for tbf */
    parent->children[0] = malloc(sizeof(struct htb_child));
    if (!parent->children[0]) {
        fprintf(stderr, "malloc(%lu) failed\n", (long)sizeof(struct
        ↪ htb_child));
        goto fail_conf;
    }

    /* using parent value */
    parent->children[0]->rate = parent->limit;
    parent->children[0]->ceil = parent->limit;
    parent->children[0]->burst = parent->burst;
    parent->children[0]->ceil_burst = parent->burst;
}
```

Nel pratico, viene creato una sola classe, con gli stessi valori del nodo padre, per gestire l'unica coda di pacchetti che servirà.

Il motivo di questa scelta è che le funzioni elencate nella sezione prima funzionano solo sui nodi figli, quindi in questo modo non è stato necessario cambiare o aggiungere codice in più.

6.5 Verifica Sperimentale

Per verificare il funzionamento del programma, verrà riproposto l'esercizio 2.3.2.

SCENARIO

Un server di un'azienda ospita tre servizi per tre clienti diversi: un servizio streaming, un sito web con molto traffico e un servizio di file hosting.

La banda totale disponibile e garantita è 100 Mbit/s. Si vuole garantire più banda al cliente del sito web rispetto agli altri due clienti. Gli altri due servizi avranno un minimo di banda garantita.

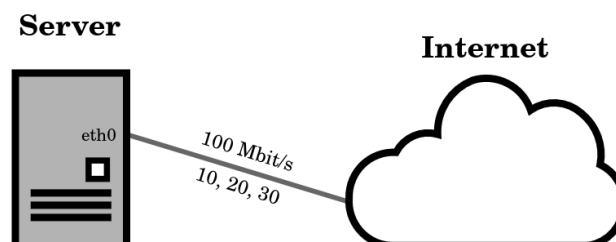


Figura 6.2: Rappresentazione grafica scenario.

In questo caso, non sarà possibile usare i class id, ma dovranno essere usati tre mark diversi: **10**, **20** e **30**.

6.5.1 Soluzione

Come ambiente di simulazione verrà usato Kathará, dove saranno presenti quattro nodi diversi: uno per simulare il server e tre per simulare i clienti. La parte riguardante l'impostazione dei file `lab.conf` e `pc<n>.startup` verrà omessa, dato che sarebbe uguale a quelle già viste.

Per prima cosa, è necessario chiarire come usare il comando `iptables` con questo nuovo programma. Non sarà più necessario solo intercettare il traffico e metterlo in una coda NFQUEUE in kernel space, ma per ogni pacchetto dovrà essere pure assegnato un mark:

BASH Regole iptables

```
# Mark 10
iptables -t mangle -A OUTPUT -p tcp --sport 80 -j MARK --set-mark 10
iptables -t mangle -A OUTPUT -p tcp --sport 443 -j MARK --set-mark 10

# Mark 20
iptables -t mangle -A OUTPUT -p tcp --sport 554 -j MARK --set-mark 20

# Mark 30
iptables -t mangle -A OUTPUT -p tcp --sport 21 -j MARK --set-mark 30

# NFQUEUE
iptables -t mangle -A OUTPUT -j NFQUEUE --queue-num 0 --queue-bypass
```

Inoltre, non sarà più possibile usare la tabella **raw**, altrimenti non verrebbe letto il mark, quindi la soluzione naturale è usare la tabella **mangle**. Successivamente, dovrà essere modificato il file `donper.conf` in modo coerente allo scenario e alle regole:

DONPER donper.conf

```
# nfqueue queue id
queue 0

# traffic limit in bits per second (suffixes K, M and G allowed) per il nodo padre
limit 100M

# bucket size parent node
burst 20K

htb yes

# classes
class mark 10 rate 50m ceil 100m burst 10k
class mark 20 rate 35m ceil 60m burst 7k
class mark 30 rate 15m ceil 20m burst 4k
```

Infine, avviando Donper con il relativo file, sarà possibile valutare i tempi.

BASH Avvio di Donper

```
donper /etc/donper/donper.conf &
```

6.5.2 Risultati

Dopo aver creato un file da 100 MB:

BASH

```
dd if=/Dev/urandom of=file.bin bs=100M count=1
```

Si possono valutare i tempi con il comando `nc` come negli esercizi degli altri capitoli.

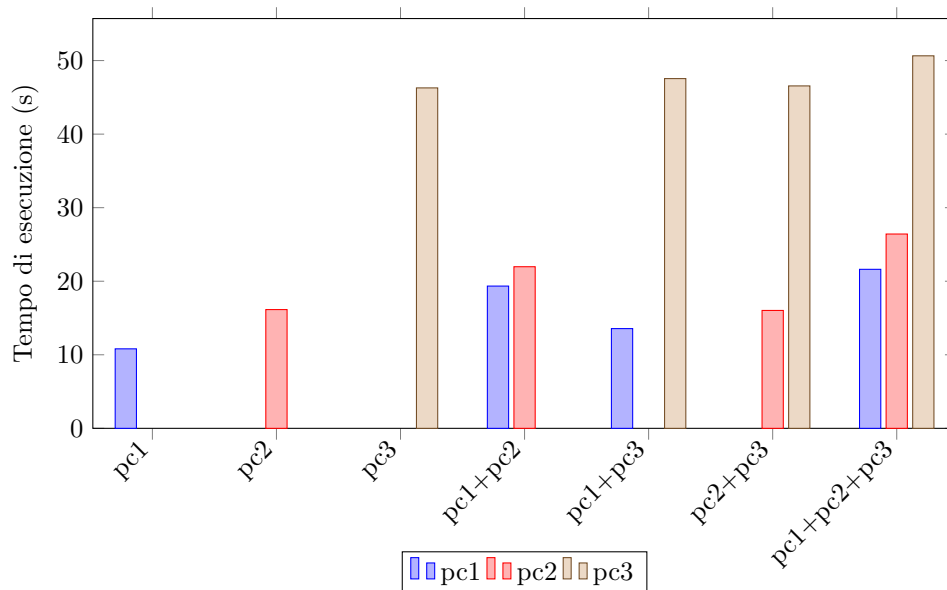
Per vedere il corretto funzionamento del programma, vanno verificate tre situazioni diverse:

1. Tutti i nodi comunicano con il server contemporaneamente.
2. Solo due nodi comunicano con il server contemporaneamente.
3. Solo un nodo comunica con il server contemporaneamente.

Per prima cosa è necessario capire i tempi ideali da confrontare:

	Ideale per Rate	Ideale per Ceil
Client 1	16.00s	8.00s
Client 2	22.86s	13.33s
Client 3	53.33s	40.00s

Successivamente, con le stesse modalità degli esercizi dei capitoli scorsi, si possono calcolare i tempi del programma Donper:



I tempi misurati si avvicinano a quelli ideali con velocità ceil quando gli altri client sono assenti, e a quelli con velocità rate quando tutti e tre sono attivi.

6.6 Considerazioni Finali

I risultati ottenuti confermano che Donper riesce a riprodurre, in userspace, un comportamento simile a quelli di HTB e TBF. I tempi calcolati rispettano il rate garantito nel caso in cui i nodi siano tutti attivi e il ceil nel caso sia possibile il prestito, aggiungendo un overhead accettabile di circa 1,5/2 secondi.

L'introduzione di una struttura gerarchica, unita all'utilizzo dell'algoritmo token bucket, risolve i principali limiti di Damper spiegati nella sezione 5.5. La possibilità di gestire più flussi tramite un solo file, senza dover usare più istanze del programma, rende anche più semplice la gestione di più classi.

Capitolo 7

Conclusione

Questo lavoro ha affrontato il problema del traffic shaping partendo dal suo funzionamento in ambienti Linux, per poi spiegarne i limiti in contesti containerizzati e, infine, introducendo due alternative interamente in user space.

Inizialmente, sono stati spiegati i concetti fondamentali sul traffic shaping in ambiente, tramite il comando `tc`, spiegando le principali discipline di accodamento Hierarchical Token Bucket e Token Bucket Filter con scenari reali di utilizzo.

Successivamente, è stato introdotto Kathará come ambiente di simulazione basato su container Docker, dimostrando, con esempi, il suo funzionamento. In questa parte, sono stati analizzati i principali limiti della containerizzazione, spiegando come il comando `tc` necessiti di moduli kernel per poter funzionare correttamente.

Dopo aver analizzato il problema, si è lavorato su un approccio diverso, spostando tutto la logica shaping del traffico dal kernel space in user space.

Il framework Netfilter, mettendo a disposizione le code NFQUEUE e il comando `iptables`, ha permesso di intercettare tutto il traffico dei processi dell'host, permettendo a un processo in stato utente di gestire la logica di shaping. In particolare, è stato scelto di usare il programma `Damper`, che tramite la libreria `libnetfilter_queue`, implementa un algoritmo leaky bucket, per rallentare il traffico in uscita secondo diversi profili.

Una volta dimostrata la fattibilità del nuovo approccio, si sono evidenziati i limiti strutturali sulla gestione di più flussi di traffico diversi, portando alla creazione di un suo fork: `Donper`. In particolare, è stato modificato l'applicativo originale, supportando la gestione di più classi di traffico e di un algoritmo di condivisione di banda inutilizzata. Inoltre, è stato sostituito l'utilizzo dell'algoritmo leaky bucket, in favore dell'algoritmo token bucket.

Nel complesso, il lavoro dimostra che è possibile realizzare un traffic shaper, con funzioni simile a quelle offerte dal kernel Linux, interamente in userspace.

Restano, però, alcuni limiti: il programma `Donper`, supporta solamente un albero a due livelli, con un nodo padre unico. Un ulteriore aspetto da considerare riguarda la scalabilità dell'approccio al crescere della velocità delle interfacce di rete. Le velocità di molte reti moderne, che possono raggiungere 2,5 o i 10 Gbit/s, potrebbero creare diversi problemi di bottleneck visto il numero di pacchetti al secondo da gestire.

Un possibile sviluppo futuro potrebbe essere quello di valutare `Donper` in contesti più complessi, migliorando la gestione dei pacchetti, limitando il numero di context switch e sfruttando meglio l'utilizzo di thread.

Bibliografia

- [1] Alexey N. Kuznetsov. *tc* — *Linux manual page*. 1999. URL: <https://man7.org/linux/man-pages/man8/tc.8.html>.
- [2] Alexey Kuznetsov. *iproute2* — *Linux manual page*. 1999. URL: <https://man7.org/linux/man-pages/man8/ip-route.8.html>.
- [3] Alexey N. Kuznetsov. *tc-tbf* — *Linux manual page*. 1999. URL: <https://man7.org/linux/man-pages/man8/tc-tbf.8.html>.
- [4] Martin Devera. *tc-htb* — *Linux manual page*. 2001. URL: <https://man7.org/linux/man-pages/man8/tc-htb.8.html>.
- [5] Martin Devera. *tc-htb* — *Htb official theory page*. 2001. URL: <http://luxik.cdi.cz/~devik/qos/htb/manual/theory.htm>.
- [6] Michael Neuling Rusty Russell. *iptables* — *Linux manual page*. 2001. URL: <https://man7.org/linux/man-pages/man8/iptables.8.html>.
- [7] Gaetano Bonofiglio et al. «Kathará: A container-based framework for implementing network function virtualization and software defined networks». In: *NOMS 2018 - 2018 IEEE/IFIP Network Operations and Management Symposium*. 2018, pp. 1–9. DOI: 10.1109/NOMS.2018.8406267.
- [8] Docker, Inc. *Docker Desktop WSL2 backend on Windows*. Docker, Inc. 2025. URL: <https://docs.docker.com/desktop/features/wsl/>.
- [9] Netfilter.org. *Netfilter.org Project*. 1999. URL: <https://www.netfilter.org/>.
- [10] Alexey Kuznetsov. *netlink* — *Linux manual page*. 1996. URL: <https://man7.org/linux/man-pages/man7/netlink.7.html>.
- [11] Netfilter.org. *libnetfilter-queue Project*. 1999. URL: https://www.netfilter.org/projects/libnetfilter_queue/doxygen/html/.
- [12] vmxdev. *Damper*. 2016. URL: <https://github.com/vmxdev/damper>.
- [13] Leonardo Donati. *Donper*. 2026. URL: <https://github.com/Donati04/donper>.